Contents

Dashboard Guide - Page by Page (with Defense Q&A) **1**
1. Home & Pipeline . . . 2
2. The Problem (PMSM & Faults) . . . 2
3. Motor & Control System . . . 3
4. Fault Detection Methods . . . 3
5. Signal Lab: Fourier vs Wavelet . . . 4
6. Scalogram Studio . . . 4
7. Dataset Explorer . . . 5
8. The CNN Model . . . 5
9. Results & Ablations . . . 6
10. Test Lab . . . 7
11. Concepts & Defense Prep . . . 7
12. Requirements Coverage . . . 8

How to use this for the defense . . . 8

Dashboard Guide - Page by Page (with Defense Q&A)

What this is. The interactive dashboard (`app.py`, launched with `./run_demo.sh`) has **12 pages** in the left sidebar. This guide explains **every page**: what it *does*, what it *contains*, a plain-language ("explain like I'm five") version, and the **defense questions** an examiner is likely to ask while that page is on screen - each with a ready answer.

> Arabic version: `docs/dashboard-guide-ar.md` (`docs/build/dashboard-guide-ar.pdf`). Use this English file and the Arabic mirror side by side.

The 12 pages

1. Home & Pipeline
2. The Problem (PMSM & Faults)
3. Motor & Control System
4. Fault Detection Methods
5. Signal Lab: Fourier vs Wavelet
6. Scalogram Studio
7. Dataset Explorer
8. The CNN Model
9. Results & Ablations
10. Test Lab
11. Concepts & Defense Prep
12. Requirements Coverage

The sidebar (always visible). Shows an **Environment** panel with [on]/[off] lights for the real manifest, current model, vibration model, and fusion model (so you instantly know what data/models are loaded), plus the student names and supervisor. If a light is [off], the relevant page falls back to documented numbers instead of crashing.

---

1. Home & Pipeline

What it does. The landing page. Gives the one-sentence project idea and the headline result, then shows the whole pipeline as six steps so the audience has a mental map before drilling in.

What it contains. - Four metric cards: **vibration balanced accuracy**, **fusion**, **current** (read live from `results/real_metrics.json`), and **3,150 scalograms** (KAIST). - The **six-step pipeline**: Signal -> Segment -> CWT -> Scalogram -> CNN -> Diagnosis. - A "what makes this project solid" strip: real data, leakage-free split, honest metrics, reproducible, Python-only, bilingual.

In simple words. This is the cover page. It says: *"We listen to a motor, turn the sound/electricity into a picture, and a computer looks at the picture and says if the motor is sick or healthy."* The big numbers at the top are our score.

Defense Q&A. - *Q: In one sentence, what did you build?* - A pipeline that converts a PMSM current/vibration signal into a wavelet scalogram image and classifies the motor's health with a CNN. - *Q: What is your headline result?* - On real KAIST data, the **vibration** channel reaches **balanced accuracy 1.00** on held-out recordings; **current** is much weaker (**0.69**). - *Q: Why six steps and not end-to-end on the raw signal?* - Each step is testable and interpretable, and turning the signal into an image lets us use CNNs, which are excellent at images.

---

2. The Problem (PMSM & Faults)

What it does. Explains *what a PMSM is*, *how it is controlled*, and *which faults* we classify - the motivation for the whole project.

What it contains. - A definition of PMSM (permanent-magnet rotor locks to the stator's rotating field, spins **synchronously**, no rotor current -> high efficiency) and **FOC** (current split into `i_d` ~= 0 for flux and `i_q` for torque). - **Four tabs**, one per class - Healthy, **InterTurn** (our main fault), Demagnetization, Overload - each with a description and a live synthetic signal plot of that condition. - A two-column "why two sensors" note: **current** is free but the controller suppresses the fault; **vibration** needs an accelerometer but shows the fault directly.

In simple words. A PMSM is a very efficient electric motor used in EVs and robots. Sometimes the wires inside it short together (**inter-turn fault**) and it can burn out. This page shows the healthy motor and the sick versions, and explains we can "listen" two ways: by its electricity (current) or by its shaking (vibration).

Defense Q&A. - *Q: What is an inter-turn fault and why is it dangerous?* - Insulation between winding turns breaks, creating a shorted loop that carries large circulating current, overheats, and can escalate to total winding failure within minutes. - *Q: Why is a PMSM "synchronous"?* - The rotor's permanent magnets lock onto the stator's rotating field and turn at exactly the field speed - no slip. - *Q: Which faults are real and which are synthetic?* - Healthy and inter-turn are from the **real** KAIST dataset; demagnetization and overload are **synthetic** (the real set doesn't contain them), and we say so honestly. - *Q: Why include current at all if it's weak?* - It's free (the FOC drive already measures it), so it's worth checking how far it can go before adding a sensor.

---

3. Motor & Control System

What it does. The electrical-machines deep-dive: how the motor is built, how it spins, how Field-Oriented Control drives it, and how it compares to its relatives. (This is the "mechatronics substance" chapter in dashboard form.)

What it contains - four tabs. - **Construction:** stator (laminated iron + 3-phase winding - *the part that fails*) and rotor (permanent magnets, no brushes), separated by the air gap; SPM vs IPM. - **Operating principle:** balanced 3-phase currents make a **rotating field** (`n_s = 120.f/P`); torque prop. to `i_q`, `i_d ~= 0`; interactive **back-EMF** (sinusoidal PMSM vs trapezoidal BLDC) and **torque-speed envelope** (constant-torque then field-weakening) plots. - **Control (FOC):** speed PI -> `i_q*`; current PI loops; Clarke + Park; inverse Park + **SVPWM** -> 3-phase inverter (6 switches). Key point: the current loop *rejects disturbances*, which **masks the fault in the current** - exactly why vibration wins. - **PMSM vs relatives:** comparison figure vs PMDC / BLDC / Induction.

In simple words. This page opens the motor up. It shows the outer coils (which can break) and the magnet center, how three electric waves make a spinning magnetic "hand" that drags the rotor around, and the smart controller that keeps it smooth. That same smart controller accidentally hides the electrical symptom of the fault - which is why shaking is a better clue than electricity.

Defense Q&A. - *Q: How does a PMSM produce a rotating field?* - Three windings 120° apart carry three currents 120° out of phase; their combined field vector rotates at `n_s = 120.f/P` rpm. - *Q: What does Field-Oriented Control actually do?* - It transforms the three phase currents into `i_d` (flux) and `i_q` (torque) via Clarke/Park, regulates them with PI loops, and reconstructs voltages with inverse Park + SVPWM, so the motor behaves like an easily-controlled DC machine. - *Q: Why does FOC hurt current-based fault detection?* - The current control loop actively suppresses deviations, so it also partly cancels the fault signature in the current - the vibration path isn't inside that loop, so it stays clean. - *Q: Why a PMSM over an induction motor here?* - Higher efficiency and torque density, no rotor current/slip; the comparison tab summarizes the trade-offs.

---

4. Fault Detection Methods

What it does. Surveys *how PMSM faults are detected today* - manual and automated - shows where each method falls short, and motivates why our CWT + CNN approach is an improvement.

What it contains. - A **detection-taxonomy** figure (the landscape of methods). - Two columns: **manual/offline** (thermal imaging, insulation resistance "Megger", surge/hi-pot - need shutdown, periodic, skilled labour) vs **online/automated** (**MCSA** current FFT + vibration FFT/envelope, model-based observers - fixed thresholds, fooled by load/speed, need to know which frequency). - The **inter-turn mechanism** figure and an interactive **MCSA spectrum** (healthy vs inter-turn - the fault raises harmonics/side-bands). - An interactive **detection-probability-vs-severity** curve showing CWT+CNN detecting *earlier* (lower severity) than a fixed-threshold FFT alarm.

In simple words. Today, technicians either stop the motor and test it by hand, or use simple alarms that watch one frequency. Those methods miss early or unusual faults, or get fooled when the motor speeds up/slows down. Our method watches the *whole* time-frequency picture and learns the pattern itself, so it catches problems sooner and isn't easily fooled.

Defense Q&A. - *Q: What is MCSA and what's its weakness?* - Motor Current Signature Analysis: take the FFT of the current and watch specific fault frequencies. Weakness: it assumes a steady operating point and a fixed threshold, so changing load/speed or a transient fault fools it. - *Q: Does your method replace the offline tests?* - No - it **complements** them. It adds a continuous, intelligent **online** monitor on sensors the drive already has; commissioning tests (hi-pot, Megger) still have their place. - *Q: Why is CWT+CNN "robust to operating point"?* - The CNN learns the fault's time-frequency texture rather than a single hand-picked frequency line, so it generalizes across load/speed better than a fixed FFT threshold.

---

5. Signal Lab: Fourier vs Wavelet

What it does. Hands-on proof of *why we use wavelets instead of plain FFT*. You build a signal and see the FFT (frequency only) next to the scalogram (frequency **and** time).

What it contains. - Controls: **condition**, **inter-turn severity**, **extra noise** sliders. - The time-domain signal plot. - Side by side: the **FFT magnitude spectrum** (fundamental at F0, faults add harmonics - but "blind to *when*") and the **CWT scalogram** image (vertical = frequency, horizontal = time, colour = energy), with a note on the uncertainty principle (`Deltat.Deltaf >= const`).

In simple words. The FFT is like a list of which musical notes are playing, but not *when*. The scalogram is like sheet music - it shows the notes *and* their timing. For a fault that comes and goes, timing matters, so the scalogram wins.

Defense Q&A. - *Q: Why not just use the FFT?* - FFT gives frequency content but throws away time information and assumes the signal is stationary. Real fault signals are **non-stationary**, so we need a joint time-frequency view. - *Q: What is the uncertainty principle here?* - You can't have perfect time and frequency resolution at once. Wavelets trade them sensibly: fine frequency resolution at low frequencies, fine time resolution at high frequencies. - *Q: What does adding noise demonstrate?* - That the scalogram's time-frequency pattern stays recognizable where a single FFT peak might get buried - robustness.

---

6. Scalogram Studio

What it does. Lets you see how **every CWT parameter** changes the final image - using the *exact* transform used in training. This is the heart of step 3 of the pipeline.

What it contains. - Knobs: **condition**, **severity**, **number of CWT scales** (16-256), **colormap**, **image size** (64-224 px), **Morlet bandwidth**. - Left: the **raw `|CWT|` coefficient matrix** (scales x time) as a heatmap. - Right: the **final RGB model-input image**,

its dimensions, and a note that scales are geometric (4->256), each mapping to a pseudo-frequency; more scales = finer vertical resolution but slower.

In simple words. This is the "photo studio" where the signal becomes a picture. You turn dials - how detailed, what colours, how big - and watch the picture change. This is exactly the picture the AI is trained on.

Defense Q&A. - *Q: What is a scalogram, precisely?* - The magnitude of the Continuous Wavelet Transform plotted as an image: one axis is the wavelet scale (~= frequency), the other is time, and the colour is the coefficient magnitude (energy). - *Q: Why the complex Morlet wavelet `cmor1.5-1.0`?* - It's well localized in both time and frequency and matches oscillatory fault signatures; the bandwidth slider shows the time-frequency trade-off. - *Q: Why 128 scales and 224x224?* - 128 scales give enough vertical (frequency) resolution without excessive cost; 224x224 is the standard CNN input size and the image-size ablation shows it's adequate. - *Q: What does changing the number of scales do?* - More scales = finer frequency detail and a taller matrix, but slower to compute; fewer scales coarsen the image.

---

7. Dataset Explorer

What it does. Shows the **real KAIST dataset** composition - how many segments per class and channel, how the leakage-free split is distributed, and lets you browse actual rendered scalograms.

What it contains. - If the manifest is present: totals (segments / channels / recordings), a **segments-per-class-and-channel** bar chart, a **segments-per-split** bar chart (leakage-free, by recording), and an **imbalance warning** (~4 healthy vs ~60 fault recordings). - If the manifest is absent (data is gitignored): a fallback table with the documented composition (current 200 H / 1350 IT, 4/27 recordings; vibration 200 H / 1400 IT, 4/28 recordings). - A **scalogram browser**: pick channel + class + count and view real images.

In simple words. This is the "ingredients list." It shows how many example pictures we have of each motor state, and warns that we have *lots* of sick examples but *very few* healthy ones - which we have to handle carefully so the AI doesn't cheat.

Defense Q&A. - *Q: What is the class imbalance and why does it matter?* - Far more fault than healthy recordings (~4 healthy). An accuracy metric would reward a model that just says "fault" every time, so we balance training and report **balanced accuracy**. - *Q: What is a leakage-free split?* - We split by **recording id**, so no two windows from the same recording land in both train and test. Otherwise near-identical windows would inflate the score. - *Q: How big is the dataset?* - 3,150 scalogram segments across current and vibration.

---

8. The CNN Model

What it does. Explains the network: its architecture, training settings, the deliberate depth choice, and the actual training curves.

What it contains. - The **architecture** (text): Input 224x224x3 -> three Conv->ReLU->MaxPool blocks (32 -> 64 -> 128 filters) -> Flatten -> Dropout(0.5) -> Dense(128) -> Dense(softmax). - Training settings: **Adam**, sparse categorical cross-entropy, **early stopping** (patience 5, restore best), random-flip augmentation, class balancing, and the fusion model's global-average-pooling head. - A **depth expander** explaining the 3-block choice with the measured depth ablation (2/3/4 blocks -> current 0.71/0.70/0.69, vibration 1.00/1.00/1.00; fewer blocks paradoxically means *more* parameters because less pooling -> bigger flatten). - A live **`model.summary()`** and **training-curve** plots (train vs val accuracy).

In simple words. This is the AI's "brain blueprint." It scans the picture in three passes (first edges, then textures, then fault shapes), then makes a decision. We picked three passes because fewer can't see enough and more just memorizes our small dataset (overfitting).

Defense Q&A. - *Q: Why three convolutional blocks?* - It's the sweet spot for a few-thousand-image dataset: enough capacity to capture fault motifs, but not so deep that it overfits; the depth ablation confirms 2/3/4 barely differ, so we take the moderate one. - *Q: Why does the 2-block net have* more *parameters than the 3-block?* - Less pooling leaves a larger feature map, so the `Flatten->Dense` layer has many more weights (~=23.9M vs ~=11.2M). - *Q: How do you fight overfitting?* - Dropout 0.5, random-flip augmentation, early stopping with best-weight restore, class balancing, and (in fusion) global average pooling instead of flatten. - *Q: Why softmax + sparse categorical cross-entropy?* - Multi-class single-label output; sparse CCE takes integer labels directly without one-hot encoding.

---

9. Results & Ablations

What it does. The evidence page: headline metrics per channel, confusion matrices, and **six ablation studies** that justify every design choice.

What it contains. - A metrics **table** (test acc, balanced acc, macro-F1, healthy recall, inter-turn recall) per channel, sorted by balanced accuracy, plus a bar chart and the **key finding** (vibration 1.00 vs current 0.69) and the **honest limitation** (only 4 healthy recordings -> can't fully exclude a recording-identity shortcut). - **Confusion matrices** and real example scalograms. - **Six ablation tabs:** 1. **Balancing** - without it, current collapses (healthy recall -> 0). 2. **Image size** - bigger helps weak current; vibration saturated. 3. **Learning curve** - vibration needs ~46 images; current is flat (signal-quality limit, not quantity). 4. **Depth (#layers)** - 2/3/4 blocks barely differ. 5. **Architecture** - **transfer learning (MobileNetV2) is the clean win**: current 0.70 -> **0.89**; the from-scratch "modern" BatchNorm+GAP variant collapses on this tiny set. 6. **Generated data** - **honest negative**: SpecAugment +0.01, synthetic pre-training *hurt* (0.70 -> 0.35, domain gap).

In simple words. This is the "report card" plus the experiments that prove our choices. The big lesson: listening to the motor's *shaking* works perfectly here; its *electricity* doesn't, and no amount of fake data fixes that - only real, varied recordings (or borrowing features from ImageNet) help.

Defense Q&A. - *Q: Vibration is 1.00 - isn't that overfitting?* - The split is leakage-free

by recording and the score is on held-out recordings. The honest caveat is that only 4 healthy recordings exist, so we present it as a strong result *with* a stated limitation, not a finished product. - *Q: Why report balanced accuracy and macro-F1, not accuracy?* - The test set is 50 healthy / 200 faulty; a model that always says "faulty" scores 80% accuracy but detects no healthy motor. Balanced accuracy and macro-F1 expose that. - *Q: Did generating data help?* - No. SpecAugment was negligible and synthetic pre-training hurt (domain gap). This is the honest result, and it contrasts with ImageNet transfer, where generic real-world features *do* transfer. - *Q: What single change improved current the most?* - **Transfer learning** with a frozen ImageNet MobileNetV2 backbone: 0.70 -> 0.89 balanced accuracy.

---

10. Test Lab

What it does. The hands-on demo: run the *real* signal -> scalogram -> CNN pipeline live, four different ways, and see the diagnosis with confidence and Grad-CAM.

What it contains - four modes. - **Synthetic generator** - choose a condition + severity + channel, generate, and diagnose. - **Real test sample** - pick a random **held-out** KAIST sample and diagnose (needs the local data). - **Upload `.npy`** - feed your own 1-D signal. - ** Quiz me** - a scalogram is shown; *you* guess, then the model guesses - can you beat the AI? - For every run: Step 1 signal plot, Step 2 scalogram, Step 3 diagnosis with a **probability bar chart**, a correct/wrong check against ground truth, and a **Grad-CAM** overlay showing *where* the CNN looked (red = most important).

In simple words. This is the "try it yourself" room. Make a motor (real or fake), press the button, and watch the AI turn it into a picture and guess the illness. The Grad-CAM is a heat-map showing which part of the picture convinced the AI. There's even a game where you race the AI.

Defense Q&A. - *Q: How do I know the demo isn't cheating?* - "Real test sample" mode draws only from **held-out** recordings the model never trained on; the prediction is computed live by the saved `.keras` model. - *Q: What is Grad-CAM and why show it?* - It highlights the time-frequency regions that most influenced the decision, letting us confirm the CNN attends to physically meaningful bands (the fault harmonics), not artefacts - model interpretability. - *Q: Why does the dashboard run on CPU?* - It only does tiny single-image inference; forcing CPU (`CUDA_VISIBLE_DEVICES=-1`) avoids competing with a training job for the 4 GB GPU's memory. - *Q: What does the confidence percentage mean?* - The softmax probability of the top class - how sure the network is, not a guarantee of correctness.

---

11. Concepts & Defense Prep

What it does. A condensed viva crib-sheet: the elevator pitch, the most common questions with crisp answers, and the numbers to memorize.

What it contains. - A **60-second elevator pitch**. - Seven **Q&A expanders**: why scalograms, why wavelets not FFT, is vibration 1.00 overfitting, why balanced accuracy, why current is weak, do you still need MATLAB, what is data leakage. - A **numbers-**

to-memorize block (dataset, sampling rates, window/overlap, 3,150 scalograms, Morlet/scales/size, split, results, tests/CI/GPU).

In simple words. This is the night-before study card. Short, punchy answers to the questions the examiners always ask, plus the key numbers you must be able to say without looking.

Defense Q&A. *(This page is itself the Q&A - the highest-value ones:) - Q: Your 60-second pitch?* - PMSMs develop inter-turn stator shorts that can destroy the winding; we window the current/vibration signal, turn each window into a wavelet scalogram, and a CNN classifies it; on real KAIST data vibration reaches balanced accuracy 1.00 on unseen recordings, current is far weaker; Python-only, reproducible, tested; we report balanced accuracy because the data is imbalanced. - *Q: Do you still need MATLAB?* - No; everything runs in Python (PyWavelets + TensorFlow). MATLAB is an optional alternative path. - *Q: The key numbers?* - KAIST `rgn5brrgrn`; current @100 kHz, vibration @25.6 kHz -> decimated to 10 kHz; 0.5 s windows, 50% overlap; 3,150 scalograms; Morlet `cmor1.5-1.0`, 128 scales, 224x224; 70/15/15 split by recording; vibration 1.00, current 0.69, fusion 0.88.

---

12. Requirements Coverage

What it does. Proves the lab satisfies **every point and keyword** of the assignment (`project-discription-ar.md`), mapping each requirement to where it is demonstrated.

What it contains. - A **coverage table**: each Arabic requirement (main task; PMSM principle; fault types; signal processing; CWT; scalogram concept; CNN basics; current/vibration; normal/overload/fault; real-or-simulated data; apply CWT; signals->images; save in class folders; read images; train; classify; extract accuracy; accuracy; confusion matrix; performance across data; effect of #images & quality; adjust #layers; change image size; improve dataset; reduce overfitting; Python/Wavelet/DL tools) -> its English meaning -> **where in this lab** it lives -> [x] status. - A **deliverables** strip: image dataset, training code (+38 tests, CI), final models, plots/results, 25-35-page report (EN+AR), 15-20 slides (EN+AR).

In simple words. This is the checklist that says: *"The assignment asked for X - here's exactly where we did X."* Every box is ticked, and each one has a live demo somewhere in the dashboard.

Defense Q&A. - *Q: Show me where requirement 6 (adjust number of layers) is satisfied.* - The **Depth ablation** (Results page, tab 4): a real 2/3/4-block study, plus the design note in The CNN Model page. - *Q: Where is "improve the dataset / reduce overfitting"?* - Balancing ablation (Results) for the dataset; dropout, augmentation, early stopping, and GAP (The CNN Model) for overfitting. - *Q: What are the formal deliverables?* - The labelled scalogram image dataset, the tested training code, the final `.keras` models, the results/plots, a 25-35-page bilingual report, and 15-20 bilingual slides - all in `docs/build/`.

How to use this for the defense

1. Open the dashboard (`./run_demo.sh`) on one screen, this guide on another.

2. Walk the pages **in order 1 -> 12** - it's a complete narrative arc (problem -> motor -> detection -> method -> data -> model -> results -> live demo -> recap -> proof).
3. For each page, lead with the **simple** explanation, then the **professional** detail if asked, and keep the **Q&A** answers ready.
4. The Test Lab (page 10) is your strongest live moment - demo the **Quiz** and **Grad-CAM** to show the model working and *why* it works.